MORENO PUENTE EMILIA CRISTINA

TALLER MINIMOS CUADRADOS

Código con la salida manual en clase

```
In [1]: import numpy as np
        import matplotlib.pyplot as plt
        import math

        a = 2.5/9
        b = 10/9

        def funcionlineal(x):
            return a*x + b

        x = np.linspace(-5, 20, 200)

        # Puntos dados
        x0, y0 = 5.0, 4.2
        x1, y1 = 10.0, 1.0
        x2, y2 = 15.0, -1.8

        plt.figure(figsize=(8,4))

        # Recta
        plt.plot(x, funcionlineal(x), label='Función lineal', color='#FF9B0F', linewidth=2)

        # Puntos
        plt.scatter(x0, y0, color='red', s=100, label='Punto 0')
        plt.scatter(x1, y1, color='orange', s=100, label='Punto 1')
        plt.scatter(x2, y2, color='yellow', s=100, label='Punto 2')

        plt.axvline(x=0, color='black', linewidth=0.8)
        plt.axhline(y=0, color='black', linewidth=0.8)

        plt.xlabel('x')
        plt.ylabel('y')
        plt.legend()
        plt.grid(True)
        plt.show()
```

Con la library

```
In [2]: import logging
        from sys import stdout
        from datetime import datetime
        logging.basicConfig(
            level=logging.INFO,
            format='%(asctime)s %(levelname)s %(message)s',
            datefmt='%H:%M:%S',
            stream=stdout
        )
        logging.info("Iniciando ejecución", {datetime.now()})

        # ============================================================
        def ajustar_mcs_cuadrados(
            mc, lista,
            lista_par=None,
            grafiquemos=False,lista2=False,list3=False), hasta=1):
            """
            ** Resuelve el sistema de ecuaciones para encontrar los parámetros del método de mínimos cuadrados. Plantea el sistema de ecuaciones lineales al reemplazar los valo

            ## Parámetros:

            `mc` : lista con los valores de x.

            `y` : lista con los valores de y.

            `grafiquemos` : Opta por las funciones de derivadas parciales de ''Par.'', [OPCIONAL]** Lista función además del de la forma: ''der_parcial_n(x,y)''.

            ## Retorna:

            `resultado` : vector con la solución del sistema de ecuaciones lineales.

            ---

            assert len(mc) == len(y), "mc y y no tienen la misma longitud."

            n = len(mc)
            log_par = len(lista_par)
            logging.info("Se encontraron %s par(es) parámetros.", log_par)
            if grafiquemos and not lista_par:
                raise ValueError("Si grafiquemos=True, lista_par es obligatorio.")
            for i, der_parcial in enumerate(lista_par):
                assert callable(der_parcial), f"Error: la derivada parcial debe ser una función."

                print(f"  der_parcial[{i}] OK", flush=True)
            print("  Todas las funciones son válidas", flush=True)
            A = [[0.0 for _ in range(n)] for _ in range(n)]
            b = [0.0 for _ in range(n)]
            print("Iniciando matrices", flush=True)
            for i in range(n):
                for j in range(n):
                    A[i][j] = sum(mc[k]**(i+j) for k in range(n))
                b[i] = sum(y[k]*mc[k]**i for k in range(n))
            print("Matrices listas", flush=True)
            print("Llamando a solve...", flush=True)
            resultado = np.linalg.solve(A, b)
            print("solve terminado", flush=True)
            logging.info("Ajuste completado. Grado %s", n-1)
            return resultado

        ajustar_mcs_cuadrados(
            [5.0, 10.0, 15.0],
            [4.2, 1.0, -1.8],
            [funcionlineal],
            True,
            [der_parcial_1],
            False,
            False,
            1)
        logging.info("Ejecución finalizada")
```

```
[22:42:22.912]INFO Iniciando ejecución, {2025-12-22 22:42:22.912767}
[22:42:22.912]INFO Se encontraron 1 par(es) parámetros.
  der_parcial[0] OK
  Todas las funciones son válidas
Iniciando matrices
Matrices listas
Llamando a solve...
solve terminado
[22:42:22.912]INFO Ajuste completado. Grado 1
[22:42:22.912]INFO Ejecución finalizada
```

```
In [3]: import numpy as np
        import matplotlib.pyplot as plt

        # ============================================================
        def ajustar_mcs_cuadrados(
            mc, lista,
            lista_par=None,
            grafiquemos=False,lista2=False,list3=False), hasta=1):
            """
            ** Resuelve el sistema de ecuaciones para encontrar los parámetros del método de mínimos cuadrados. Plantea el sistema de ecuaciones lineales al reemplazar los valo

            ## Parámetros:

            `mc` : lista con los valores de x.

            `y` : lista con los valores de y.

            `grafiquemos` : Opta por las funciones de derivadas parciales de ''Par.'', [OPCIONAL]** Lista función además del de la forma: ''der_parcial_n(x,y)''.

            ## Retorna:

            `resultado` : vector con la solución del sistema de ecuaciones lineales.

            ---

            assert len(mc) == len(y), "mc y y no tienen la misma longitud."

            n = len(mc)
            log_par = len(lista_par)
            logging.info("Se encontraron %s par(es) parámetros.", log_par)
            if grafiquemos and not lista_par:
                raise ValueError("Si grafiquemos=True, lista_par es obligatorio.")
            for i, der_parcial in enumerate(lista_par):
                assert callable(der_parcial), f"Error: la derivada parcial debe ser una función."

                print(f"  der_parcial[{i}] OK", flush=True)
            print("  Todas las funciones son válidas", flush=True)
            A = [[0.0 for _ in range(n)] for _ in range(n)]
            b = [0.0 for _ in range(n)]
            print("Iniciando matrices", flush=True)
            for i in range(n):
                for j in range(n):
                    A[i][j] = sum(mc[k]**(i+j) for k in range(n))
                b[i] = sum(y[k]*mc[k]**i for k in range(n))
            print("Matrices listas", flush=True)
            print("Llamando a solve...", flush=True)
            resultado = np.linalg.solve(A, b)
            print("solve terminado", flush=True)
            logging.info("Ajuste completado. Grado %s", n-1)
            return resultado

        ajustar_mcs_cuadrados(
            [5.0, 10.0, 15.0],
            [4.2, 1.0, -1.8],
            [funcionlineal],
            True,
            [der_parcial_1],
            False,
            False,
            1)
```

```
[22:42:22.912]INFO Iniciando ejecución, {2025-12-22 22:42:22.912767}
[22:42:22.912]INFO Se encontraron 1 par(es) parámetros.
  der_parcial[0] OK
  Todas las funciones son válidas
Iniciando matrices
Matrices listas
Llamando a solve...
solve terminado
[22:42:22.912]INFO Ajuste completado. Grado 1
[22:42:22.912]INFO Ejecución finalizada
```

```
In [4]: import numpy as np

        # ============================================================
        def ajustar_mcs_cuadrados(
            mc, lista,
            lista_par=None,
            grafiquemos=False,lista2=False,list3=False), hasta=1):
            """
            ** Resuelve el sistema de ecuaciones para encontrar los parámetros del método de mínimos cuadrados. Plantea el sistema de ecuaciones lineales al reemplazar los valo

            ## Parámetros:

            `mc` : lista con los valores de x.

            `y` : lista con los valores de y.

            `grafiquemos` : Opta por las funciones de derivadas parciales de ''Par.'', [OPCIONAL]** Lista función además del de la forma: ''der_parcial_n(x,y)''.

            ## Retorna:

            `resultado` : vector con la solución del sistema de ecuaciones lineales.

            ---

            assert len(mc) == len(y), "mc y y no tienen la misma longitud."

            n = len(mc)
            log_par = len(lista_par)
            logging.info("Se encontraron %s par(es) parámetros.", log_par)
            if grafiquemos and not lista_par:
                raise ValueError("Si grafiquemos=True, lista_par es obligatorio.")
            for i, der_parcial in enumerate(lista_par):
                assert callable(der_parcial), f"Error: la derivada parcial debe ser una función."

                print(f"  der_parcial[{i}] OK", flush=True)
            print("  Todas las funciones son válidas", flush=True)
            A = [[0.0 for _ in range(n)] for _ in range(n)]
            b = [0.0 for _ in range(n)]
            print("Iniciando matrices", flush=True)
            for i in range(n):
                for j in range(n):
                    A[i][j] = sum(mc[k]**(i+j) for k in range(n))
                b[i] = sum(y[k]*mc[k]**i for k in range(n))
            print("Matrices listas", flush=True)
            print("Llamando a solve...", flush=True)
            resultado = np.linalg.solve(A, b)
            print("solve terminado", flush=True)
            logging.info("Ajuste completado. Grado %s", n-1)
            return resultado

        ajustar_mcs_cuadrados(
            [5.0, 10.0, 15.0],
            [4.2, 1.0, -1.8],
            [funcionlineal],
            True,
            [der_parcial_1],
            False,
            False,
            1)
```

```
[22:42:22.912]INFO Iniciando ejecución, {2025-12-22 22:42:22.912767}
[22:42:22.912]INFO Se encontraron 1 par(es) parámetros.
  der_parcial[0] OK
  Todas las funciones son válidas
Iniciando matrices
Matrices listas
Llamando a solve...
solve terminado
[22:42:22.912]INFO Ajuste completado. Grado 1
[22:42:22.912]INFO Ejecución finalizada
```

Con la clase

```
In [5]: def par_parcial_n(x: list, y: list):
            '''
            ** Calcula la derivada parcial respecto del parámetro n al reemplazar los valores ''mc'' y ''yc''. La ecuación de la forma:
            ''f_n = f_n(x,y)''.

            ## Parámetros:

            `mc` : lista de valores de x.

            `yc` : lista de valores de y.

            ## Retorna:

            `f_n` : coeficiente del parámetro n.

            `f_n0` : coeficiente del término independiente.

            ---

            # coeficientes del término independiente
            f_n0 = sum(y)

            # coeficientes del parámetro 1
            f_n = sum(x*y)

            return [f_n, f_n0]

        def par_parcial_2(x: list, y: list):
            '''
            ** Calcula la derivada parcial respecto del parámetro 2 al reemplazar los valores ''mc'' y ''yc''. La ecuación de la forma:
            ''f_n = f_n(x,y)''.

            ## Parámetros:

            `mc` : lista de valores de x.

            `yc` : lista de valores de y.

            ## Retorna:

            `f_n` : coeficiente del parámetro 1.

            `f_n0` : coeficiente del parámetro 0.

            `f_n2` : coeficiente del término independiente.

            ---

            # coeficientes del término independiente
            f_n2 = sum(y)

            # coeficientes del parámetro 1
            f_n = sum(x*y)

            # coeficientes del parámetro 2
            f_n0 = sum(x**2*y)

            return [f_n, f_n0, f_n2]

        n, b = ajustar_mcs_cuadrados(
            mc=[5.0, 10.0, 15.0], y=[4.2, 1.0, -1.8], grafiquemos=[der_parcial_1], der_parcial_0)

        print("f_n: {n}, b: {b}")

        xs = [5.0, 10.0, 15.0]
        ys = [4.2, 1.0, -1.8]

        par = par_parcial_2(xs, ys)

        # Plot the original data points
        plt.scatter(xs, ys, color='purple', label='Data points', s=100)

        # Plot the fitted line
        x_line = np.linspace(min(xs), max(xs), 100)
        y_line = [n * x + b for x in x_line]
        plt.plot(x_line, y_line, color='#FF9B0F', label=f'y = {n:.2f}x + {b:.2f}')

        plt.xlabel('x')
        plt.ylabel('y')
        plt.title('Linear Regression')
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.show()
```

```
[22:42:22.912]INFO Iniciando ejecución, {2025-12-22 22:42:22.912767}
n: 0.4570000000000001, b: 0.8370000000000011
```